CODEALPHA

PYTHON PROGRAMMING INTERNSHIP

PROJECT REPORT

TASK 1 : HANGMAN GAME

A GUI-Based Word Guessing Game Using Python

Submitted By:

Pooja Subhasa Kenganingappanavar

Branch:

Computer Science Engineering

Internship Domain:

Python Programming

Organization:

CodeAlpha

Academic Year:

2026-2027

Project Type:

Internship Task – 1

HANGMAN GAME

DECLARATION

I hereby declare that the project entitled **“Hangman Game”** has been developed as part of the CodeAlpha Python Programming Internship – Task 1.

The project is a graphical Hangman game that provides an interactive interface for entering the player's name, starting the game, selecting letters, using hints, tracking lives, time and score, and displaying the final result.

The work presented in this report is prepared for educational and internship purposes and represents the implementation and testing of the Hangman game shown in the attached output screenshots.

Name:	Pooja Subhasa Kenganingappanavar
Branch:	Computer Science Engineering
Internship:	CodeAlpha Python Programming Internship
Date:	13-08-2026
Place:	Bengaluru
Signature:	_psk_____

TABLE OF CONTENTS

1. INTRODUCTION

Hangman is a word-guessing game in which the player tries to identify a hidden word by selecting letters.
This project presents a graphical version of the game with a simple and user-friendly interface.

The application starts with a player-name screen. After starting the game, the player sees hidden letters, a keyboard of alphabet buttons, a hint option, and game information such as player name, time, score and lives.

The screenshots show the complete user flow from entering the name to starting a round, using a hint, and successfully completing the word.

2. PROBLEM STATEMENT

The project aims to develop a simple interactive Hangman game that is easy to play and demonstrates basic Python programming and GUI concepts. The application should accept a player name, display a hidden word, allow letter-by-letter guessing, provide feedback, manage lives and score, and show a clear win or game-result message.

3. OBJECTIVES

• To develop an interactive Hangman game using Python.

• To provide a graphical user interface for the player.

• To allow the player to enter a name before starting the game.

• To display a hidden word and allow letter-by-letter guessing.

• To provide alphabet buttons for easy input.

• To provide a hint option during gameplay.

• To track time, score and remaining lives.

• To display the final result clearly after the word is completed.

• To practice conditions, loops, functions, variables and event-driven GUI programming.

• To improve logical thinking and problem-solving skills.

4. SCOPE OF THE PROJECT

The current project focuses on a single-player Hangman game with a graphical interface. The screenshots demonstrate a name-entry screen, game screen, hint display and successful completion screen.

The project can be extended later with larger word collections, difficulty levels, persistent scores, categories, sounds and additional game modes.

5. TECHNOLOGIES AND TOOLS USED

5.1 Python

Python is used as the main programming language for implementing the game logic and application behaviour.

5.2 CustomTkinter / GUI Components

The graphical interface provides windows, labels, input fields and buttons for player interaction. The screenshots show a modern dark-themed interface.

5.3 Random Word Selection

A word can be selected from the available word collection for each round.

5.4 Visual Studio Code

A Python development environment such as Visual Studio Code can be used to write, edit and run the application.

5.5 Windows

The provided screenshots show the application running on a Windows desktop.

6. PROJECT DESCRIPTION

The Hangman application begins with a welcome screen titled “Hangman”. The player enters a name and selects the “Start Game” button.

After the game starts, the interface displays the player name, a timer, score and remaining lives at the top.
The hidden word is shown using blank spaces and an alphabet keyboard is provided below it.

The player can select letters using the on-screen buttons. A hint can also be requested. In the shown example, the hint identifies the word as a citrus fruit. After the correct letters are selected, the complete word is displayed and the application shows a “YOU WIN!” message.

This design makes the game more visual and interactive than a basic console implementation.

7. FEATURES OF THE APPLICATION

- Player name entry screen
- Start Game button
- Hidden word display
- On-screen A–Z keyboard
- Hint feature with life deduction
- Timer display
- Score tracking
- Life/heart indicator
- Win message
- Clean dark-themed graphical interface

8. METHODOLOGY

The Hangman game was developed through a sequence of simple stages:

Step 1: Define the game requirements and user flow.

Step 2: Create the graphical start screen.

Step 3: Accept and store the player's name.

Step 4: Start a new game round.

Step 5: Select a word and hide its letters.

Step 6: Display the alphabet buttons for guesses.

Step 7: Check each selected letter against the target word.

Step 8: Update the displayed word, score and lives.

Step 9: Provide a hint when requested and update the life count.

Step 10: Check the winning/game-ending condition and display the result.

9. GAME WORKFLOW

START → Enter Player Name → Start Game → Select/Hide Word → Display Keyboard → Guess Letter → Check Letter → Update Word/Score/Lives → Check Result → Continue or Display WIN →END

10. ALGORITHM

1. Start the application.
2. Display the Hangman welcome screen.
3. Accept the player's name.
4. Start the game when the Start Game button is selected.
5. Choose a word for the round.
6. Display the word using hidden characters.
7. Display the alphabet buttons and game statistics.
8. Wait for the player to select a letter.
9. Check whether the selected letter occurs in the word.
10. If correct, reveal the letter and update the score.
11. If incorrect, update the remaining lives.
12. If the hint is used, display the clue and reduce a life as shown in the interface.
13. Check whether all letters have been revealed.
14. If the word is completed, display the winning message.
15. Otherwise, continue accepting guesses.
16. End the round.

11.PYTHON CONCEPTS USED

Variables

Used to store player information, selected word, score, lives, timer values and game state.

Strings

Used for words, letters, player names and messages displayed in the application.

Lists

Useful for storing a collection of possible words or letters.

If-Else Statements

Used to make decisions such as correct/incorrect guesses and win conditions.

Loops / Repeated Events

Used to continue gameplay and update the game state until the round ends.

Functions

Used to organize game actions such as starting a game, checking guesses and displaying results.

GUI Event Handling

Button clicks and user actions trigger the corresponding game operations.

Random Selection

Can be used to choose a word for each game round.

12. IMPLEMENTATION

The implementation combines Python game logic with a graphical user interface. The first screen collects the player name and starts the game. The game screen contains the hidden word area, game statistics, hint button and alphabet keyboard.

Each alphabet button represents a possible guess. When a player selects a letter, the program checks the selected word and updates the visible letters and game statistics.

The interface also provides feedback through the hint area and the final result message. In the supplied successful-game screenshot, the word “orange” is fully displayed and the application shows “YOU WIN!”.

The implementation is designed to keep the gameplay simple while providing a more attractive and interactive user experience.

13.TESTING AND RESULTS

Test Case	Action	Expected Result	Result
1	Open application	Welcome screen appears	Pass
2	Enter player name	Name is accepted	Pass
3	Start Game	Game screen appears	Pass
4	Select a correct letter	Letter is revealed	Pass
5	Use hint	Hint is displayed and life changes	Pass
6	Complete the word	Word is displayed and win message appear	Pass

14.OUTPUT SCREENSHOTS

14.1 Game Starting Screen

The first screenshot shows the initial Hangman welcome screen. The application displays the Hangman title, a field for entering the player's name, and the Start Game button.

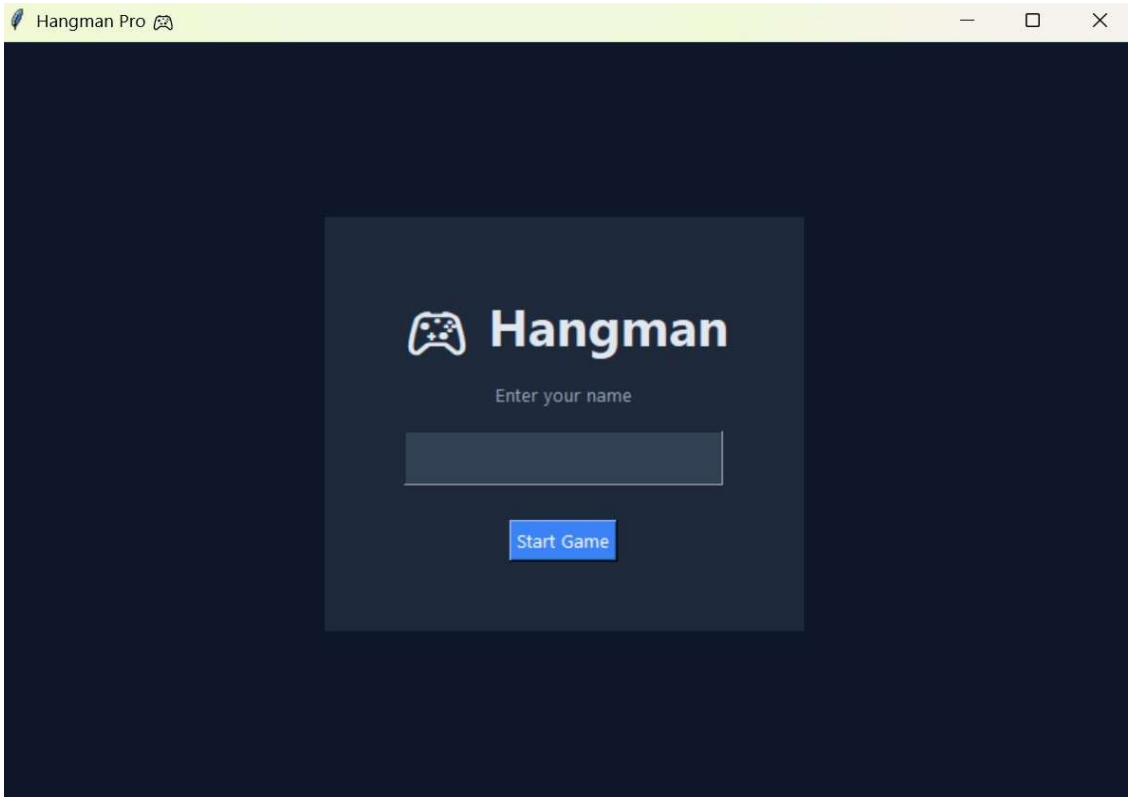

Figure 1: Hangman Game – Initial Welcome Screen

14.2 Player Name Entry

The second screenshot shows the player name entered as “ABC”. The Start Game button is ready to launch the game.

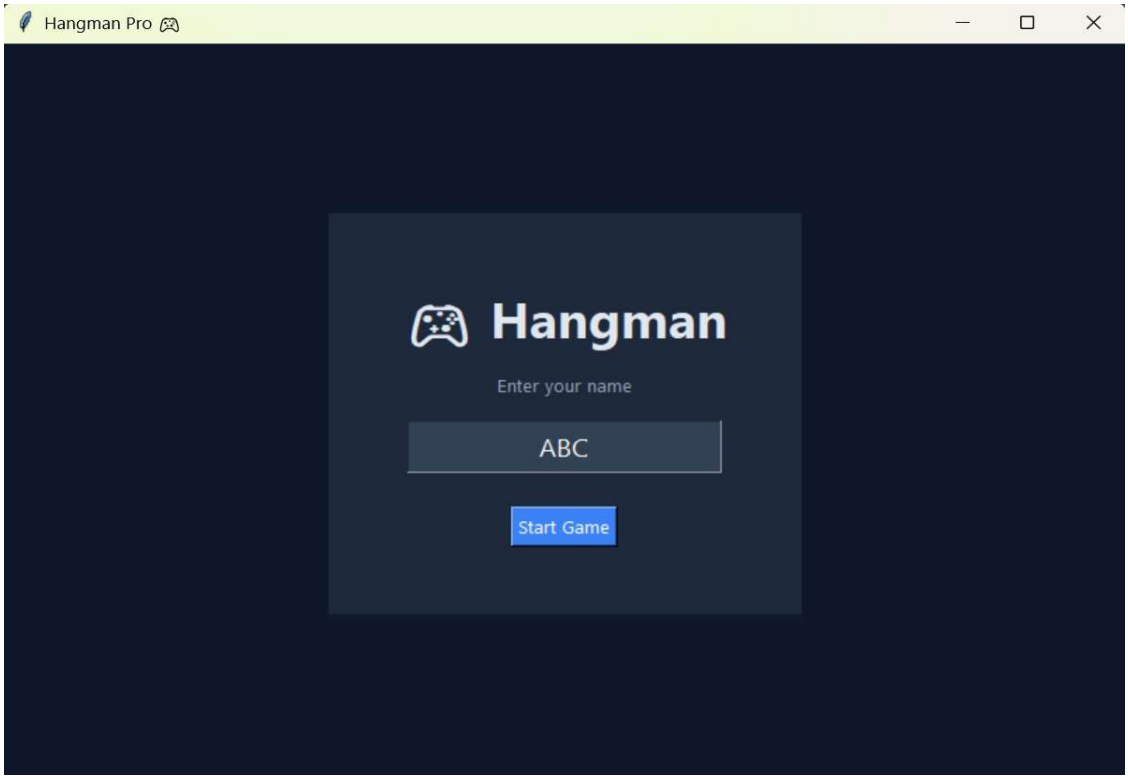

Figure 2: Player Name Entry Screen

14.3 Game Screen with Hidden Word

The third screenshot shows the main gameplay screen. The word is hidden, the alphabet keyboard is displayed, and the top section shows the player name, timer, score and remaining lives.

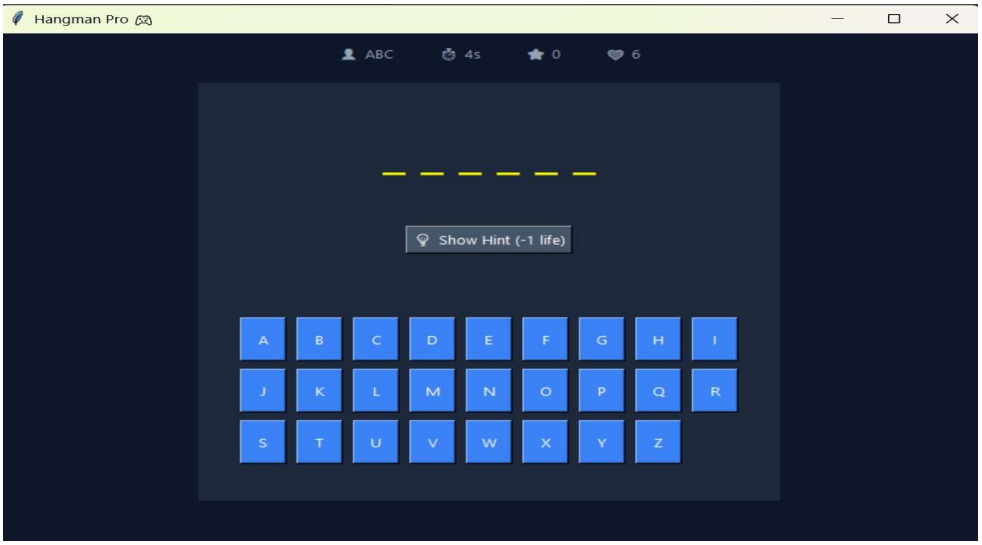

Figure 3: Main Hangman Gameplay Screen

14.4 Hint Display

The fourth screenshot shows the hint feature. The hint displayed is “A citrus fruit”. The interface also shows the updated game statistics after using the hint.

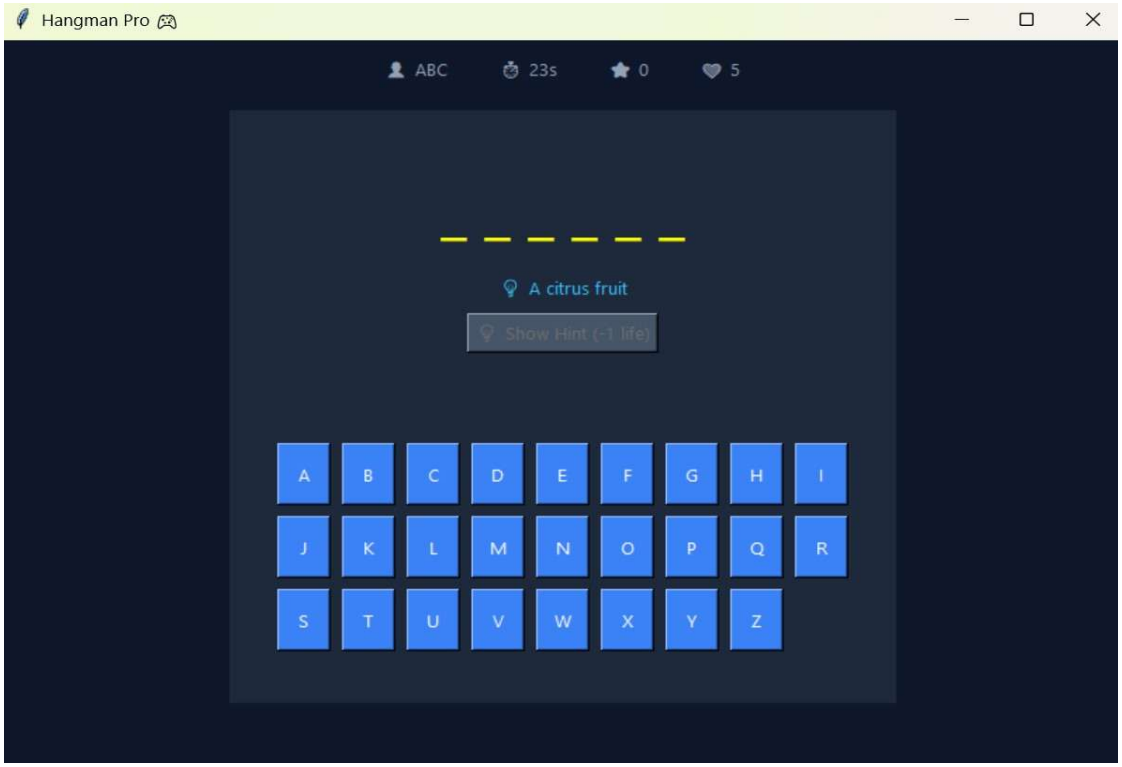

Figure 4: Hint Feature During Gameplay

14.5 Successful Game Result

The fifth screenshot shows the completed word “orange”. The hint remains visible and the application displays the message “YOU WIN!”, confirming successful completion of the round.

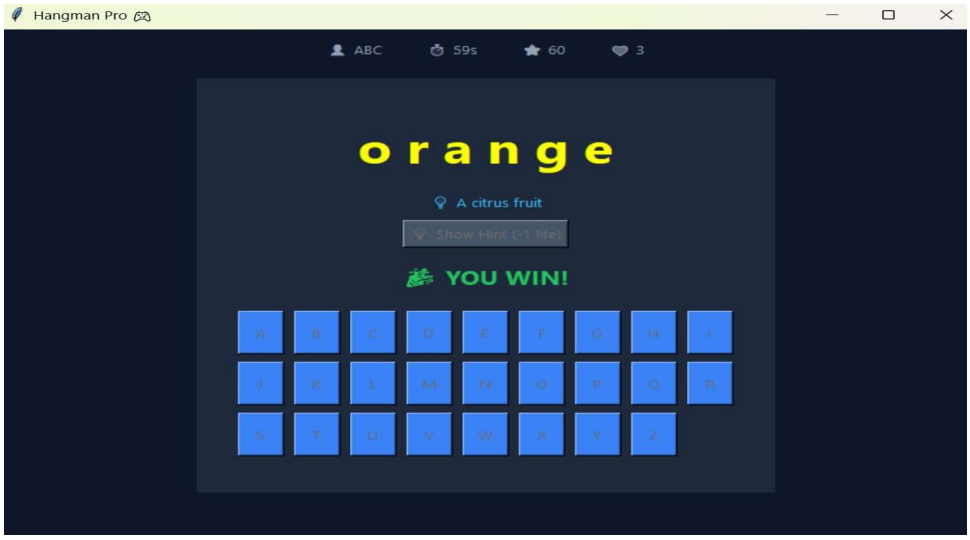

Figure 5: Successful Game Completion – YOU WIN!

15. ADVANTAGES AND LIMITATIONS

Advantages

- Simple and easy-to-use graphical interface.
- Interactive alphabet buttons make guessing convenient.
- Includes timer, score and lives for a game-like experience.
- Hint feature provides additional assistance.
- Provides clear feedback when the player wins.
- Useful for practicing Python programming and GUI event handling.

Limitations

- The game depends on the available word collection.
- The current interface is designed for a single player.
- The screenshots demonstrate a limited set of game states.
- A persistent online leaderboard or database is not shown.

16. FUTURE ENHANCEMENTS

- Add more words and word categories.
- Introduce Easy, Medium and Hard difficulty levels.
- Add a high-score or leaderboard system.
- Add sound effects and animations.
- Store game history and scores.
- Add multiple game modes or multiplayer support.
- Improve accessibility and keyboard controls.

17. CONCLUSION

The Hangman Game was successfully developed as a CodeAlpha Python Programming Internship Task. The provided screenshots demonstrate a complete graphical user flow: entering the player's name, starting the game, viewing the hidden word, using the hint feature, making guesses and reaching a successful “YOU WIN!” result.

The project demonstrates practical use of Python programming, GUI components, event-driven interaction, conditional logic, variables, strings, lists and game-state management. It also provides a foundation for adding more advanced features in future versions.